

PHP Procedural CRUD Project

MySQL • Bootstrap • Image Upload • Gender & Hobbies • State-City Dependent
Dropdown (AJAX)

A complete, beginner-friendly reference build: plain procedural PHP handles Create, Read, Update and Delete, while jQuery AJAX is used for exactly one thing — loading cities when a state is selected.

1. Project Structure

The project is organized as a small, flat PHP application with a single uploads folder for images and a js folder for the one piece of client-side AJAX code.

```
php-crud/
|
|-- index.php
|-- add.php
|-- edit.php
|-- delete.php
|-- cities.php
|-- db.php
|
|-- uploads/
|
|-- js/
    |-- script.js
```

2. Database Setup

2.1 Create the Database

```
CREATE DATABASE php_crud;

USE php_crud;
```

2.2 States Table

```
CREATE TABLE states (
    id INT AUTO_INCREMENT PRIMARY KEY,
    state_name VARCHAR(100) NOT NULL
);
```

2.3 Cities Table

```
CREATE TABLE cities (
    id INT AUTO_INCREMENT PRIMARY KEY,
    city_name VARCHAR(100) NOT NULL,
    state_id INT NOT NULL
);
```

2.4 Users Table

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL,
    image VARCHAR(255),
    gender VARCHAR(20),
    hobbies VARCHAR(255),
    state_id INT,
    city_id INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2.5 Sample States

```
INSERT INTO states (state_name) VALUES
('Delhi'),
('Maharashtra'),
```

```
('Uttar Pradesh'),
('Rajasthan'),
('Gujarat');
```

2.6 Sample Cities

```
INSERT INTO cities (city_name, state_id) VALUES
('New Delhi', 1),
('North Delhi', 1),
('South Delhi', 1),

('Mumbai', 2),
('Pune', 2),
('Nagpur', 2),

('Lucknow', 3),
('Kanpur', 3),
('Noida', 3),

('Jaipur', 4),
('Jodhpur', 4),
('Udaipur', 4),

('Ahmedabad', 5),
('Surat', 5),
('Vadodara', 5);
```

The important relationship is states.id to cities.state_id. For example, Maharashtra (id = 2) is the parent of Mumbai, Pune, and Nagpur.

```
states.id
  |
  v
cities.state_id

Maharashtra (id = 2)
  |
  +-- Mumbai
  +-- Pune
  `-- Nagpur
```

3. Database Connection — db.php

db.php

```
<?php

$conn = mysqli_connect(
    "localhost",
    "root",
    "",
    "php_crud"
);

if (!$conn) {
    die("Database connection failed: " . mysqli_connect_error());
}

mysqli_set_charset($conn, "utf8mb4");

?>
```

Note: If your MySQL username or password is different, change the "root" and "" values in the mysqli_connect() call.

4. Main Page — index.php

index.php

This is the main page of the application. It shows the Add User form, lists all existing users, loads the list of states, and uses AJAX only for populating the city dropdown.

```
<?php

include "db.php";

$states = mysqli_query(
    $conn,
    "SELECT * FROM states ORDER BY state_name"
);

$result = mysqli_query(
    $conn,
    "SELECT
        users.*,
        states.state_name,
        cities.city_name
    FROM users
    LEFT JOIN states
        ON users.state_id = states.id
    LEFT JOIN cities
        ON users.city_id = cities.id
    ORDER BY users.id DESC"
);

?>
```

4.1 Success / Error Message

```
<?php if (isset($_GET['msg'])) { ?>
    <div class="alert alert-success">
        <?php echo htmlspecialchars($_GET['msg']); ?>
    </div>
<?php } ?>
```

4.2 Add User Form (excerpt)

The form posts directly to add.php using a normal HTML form submission (no AJAX). It includes fields for name, email, image upload, gender (radio buttons), hobbies (checkboxes), and the state/city dropdowns.

```
<form action="add.php" method="POST" enctype="multipart/form-data">

    <input type="text" name="name" class="form-control">
    <input type="email" name="email" class="form-control">
    <input type="file" name="image" class="form-control" accept="image/*">

    <input type="radio" name="gender" value="Male" class="form-check-input">
    <input type="radio" name="gender" value="Female" class="form-check-input">

    <input type="checkbox" name="hobbies[]" value="Reading" class="form-check-input">
    <input type="checkbox" name="hobbies[]" value="Music" class="form-check-input">
    <input type="checkbox" name="hobbies[]" value="Sports" class="form-check-input">
```

```

☐

<select name="state_id" id="state_id" class="form-select">
  <option value="">Select State</option>
  <?php while ($state = mysqli_fetch_assoc($states)) { ?>
    <option value="<?php echo $state['id']; ?>">
      <?php echo htmlspecialchars($state['state_name']); ?>
    </option>
  <?php } ?>
</select>

<select name="city_id" id="city_id" class="form-select">
  <option value="">Select City</option>
</select>

<button type="submit" class="btn btn-primary">Save</button>
</form>

```

4.3 Users List Table (excerpt)

The table lists every user with their photo, gender, hobbies, state/city names, and Edit / Delete action buttons.

```

<?php while ($row = mysqli_fetch_assoc($result)) { ?>
<tr>
  <td><?php echo $row['id']; ?></td>
  <td>
    <?php if (!empty($row['image'])) { ?>
      
    <?php } else { ?>
      No Image
    <?php } ?>
  </td>
  <td><?php echo htmlspecialchars($row['name']); ?></td>
  <td><?php echo htmlspecialchars($row['email']); ?></td>
  <td><?php echo htmlspecialchars($row['gender']); ?></td>
  <td><?php echo htmlspecialchars($row['hobbies']); ?></td>
  <td><?php echo htmlspecialchars($row['state_name']); ?></td>
  <td><?php echo htmlspecialchars($row['city_name']); ?></td>
  <td>
    <a href="edit.php?id=<?php echo $row['id']; ?>" class="btn btn-sm btn-warning">Edit</a>
    <a href="delete.php?id=<?php echo $row['id']; ?>" class="btn btn-sm btn-danger"
      onclick="return confirm('Are you sure you want to delete?');">Delete</a>
  </td>
</tr>
<?php } ?>

```

4.4 Scripts loaded at the bottom of index.php

```

<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="js/script.js"></script>

```

5. Create — add.php

add.php

Plain procedural PHP — no AJAX. Reads form values, runs minimal validation, escapes strings, handles the image upload, joins the hobbies array into a comma-separated string, and inserts the new row.

```
<?php

include "db.php";

$name      = trim($_POST['name'] ?? '');
$email     = trim($_POST['email'] ?? '');
$gender    = $_POST['gender'] ?? '';
$state_id  = (int)($_POST['state_id'] ?? 0);
$city_id   = (int)($_POST['city_id'] ?? 0);

$hobbies = '';
if (isset($_POST['hobbies'])) {
    $hobbies = implode(',', $_POST['hobbies']);
}

if ($name == '') { die("Name is required"); }
if ($email == '') { die("Email is required"); }

$name      = mysqli_real_escape_string($conn, $name);
$email     = mysqli_real_escape_string($conn, $email);
$gender    = mysqli_real_escape_string($conn, $gender);
$hobbies   = mysqli_real_escape_string($conn, $hobbies);

$imageName = '';
if (isset($_FILES['image']) && $_FILES['image']['error'] == 0) {
    $imageName = time() . '_' . basename($_FILES['image']['name']);
    $uploadPath = "uploads/" . $imageName;
    move_uploaded_file($_FILES['image']['tmp_name'], $uploadPath);
}

$sql = "INSERT INTO users
        (name, email, image, gender, hobbies, state_id, city_id)
        VALUES
        ('$name', '$email', '$imageName', '$gender', '$hobbies', $state_id, $city_id)";

if (mysqli_query($conn, $sql)) {
    header("Location: index.php?msg=User added successfully");
    exit;
} else {
    die("Database Error: " . mysqli_error($conn));
}

?>
```

6. Update — edit.php

edit.php

edit.php does double duty: it processes the update when the form is posted, and it renders the pre-filled edit form when loaded with a GET id.

6.1 Handling the update (POST)

```

<?php

include "db.php";

$id = (int)($_GET['id'] ?? 0);
if ($id <= 0) { die("Invalid ID"); }

if (isset($_POST['save'])) {

    $name      = trim($_POST['name'] ?? '');
    $email      = trim($_POST['email'] ?? '');
    $gender     = $_POST['gender'] ?? '';
    $state_id   = (int)($_POST['state_id'] ?? 0);
    $city_id    = (int)($_POST['city_id'] ?? 0);

    $hobbies    = '';
    if (isset($_POST['hobbies'])) {
        $hobbies = implode(',', $_POST['hobbies']);
    }

    if ($name == '') { die("Name is required"); }
    if ($email == '') { die("Email is required"); }

    $name      = mysqli_real_escape_string($conn, $name);
    $email      = mysqli_real_escape_string($conn, $email);
    $gender     = mysqli_real_escape_string($conn, $gender);
    $hobbies    = mysqli_real_escape_string($conn, $hobbies);

    // Get old image so it can be replaced/deleted
    $oldResult = mysqli_query($conn, "SELECT image FROM users WHERE id=$id");
    $oldUser   = mysqli_fetch_assoc($oldResult);
    if (!$oldUser) { die("User not found"); }
    $oldImage  = $oldUser['image'];

    $imageSQL = "";
    if (isset($_FILES['image']) && $_FILES['image']['error'] == 0) {
        $newImage = time() . '_' . basename($_FILES['image']['name']);
        $uploadPath = "uploads/" . $newImage;

        if (move_uploaded_file($_FILES['image']['tmp_name'], $uploadPath)) {
            if (!empty($oldImage) && file_exists("uploads/" . $oldImage)) {
                unlink("uploads/" . $oldImage);
            }
            $imageSQL = ", image='$newImage'";
        }
    }

    $sql = "UPDATE users SET
            name='$name', email='$email', gender='$gender',
            hobbies='$hobbies', state_id=$state_id, city_id=$city_id
            $imageSQL
            WHERE id=$id";

    if (mysqli_query($conn, $sql)) {
        // IMPORTANT: no HTML/output before this header() call.
        header("Location: index.php?msg=User updated successfully");
        exit;
    } else {
        die("Database Error: " . mysqli_error($conn));
    }
}

?>

```

6.2 Loading the existing user, states, and dependent cities

```
<?php

$sql    = "SELECT * FROM users WHERE id=$id";
$result = mysqli_query($conn, $sql);
$user   = mysqli_fetch_assoc($result);
if (!$user) { die("User not found"); }

$states = mysqli_query($conn, "SELECT * FROM states ORDER BY state_name");

$cities = [];
if (!empty($user['state_id'])) {
    $state_id = (int)$user['state_id'];
    $cityResult = mysqli_query(
        $conn,
        "SELECT * FROM cities WHERE state_id=$state_id ORDER BY city_name"
    );
    while ($city = mysqli_fetch_assoc($cityResult)) {
        $cities[] = $city;
    }
}

$userHobbies = [];
if (!empty($user['hobbies'])) {
    $userHobbies = explode(',', $user['hobbies']);
}

?>
```

6.3 Pre-filled form (excerpt)

The form re-uses the same field layout as index.php, but every field is pre-populated: text inputs use `value="..."`, the matching gender radio gets checked, each hobby checkbox is checked with `in_array()`, and both the state and the already-loaded city are pre-selected.

```
<input type="text" name="name" class="form-control"
value="<?php echo htmlspecialchars($user['name']); ?>">

<input type="radio" name="gender" value="Male" class="form-check-input"
<?php if ($user['gender'] == 'Male') { echo 'checked'; } ?>>

<?php foreach ($hobbyList as $hobby) { ?>
    <input type="checkbox" name="hobbies[]" value="<?php echo $hobby; ?>"
    <?php if (in_array($hobby, $userHobbies)) { echo 'checked'; } ?>>
<?php } ?>

<select name="state_id" id="state_id" class="form-select">
    <?php while ($state = mysqli_fetch_assoc($states)) { ?>
        <option value="<?php echo $state['id']; ?>"
            <?php if ($state['id'] == $user['state_id']) { echo 'selected'; } ?>>
            <?php echo htmlspecialchars($state['state_name']); ?>
        </option>
    <?php } ?>
</select>

<select name="city_id" id="city_id" class="form-select">
    <?php foreach ($cities as $city) { ?>
        <option value="<?php echo $city['id']; ?>"
            <?php if ($city['id'] == $user['city_id']) { echo 'selected'; } ?>>
            <?php echo htmlspecialchars($city['city_name']); ?>
        </option>
    <?php } ?>
```

</select>

Note: For a cleaner production version, separate the update-processing logic from the HTML in edit.php. Keeping both in one file is fine for learning, but splitting them avoids the "no output before header()" trap as the file grows.

7. Delete — delete.php

delete.php

Also plain PHP, triggered by a normal link with a confirm() dialog in the browser — not AJAX. It removes the database row and then deletes the associated image file from disk.

```
<?php

include "db.php";

$id = (int)($_GET['id'] ?? 0);
if ($id <= 0) { die("Invalid ID"); }

$result = mysqli_query($conn, "SELECT image FROM users WHERE id=$id");
$user    = mysqli_fetch_assoc($result);
if (!$user) { die("User not found"); }

$image = $user['image'];

$sql = "DELETE FROM users WHERE id=$id";

if (mysqli_query($conn, $sql)) {
    if (!empty($image) && file_exists("uploads/" . $image)) {
        unlink("uploads/" . $image);
    }
    header("Location: index.php?msg=User deleted successfully");
    exit;
} else {
    die("Delete failed: " . mysqli_error($conn));
}

?>
```

8. AJAX Endpoint — cities.php

cities.php

This is the only PHP file called through AJAX in the whole project. It receives a state_id via POST and returns a plain HTML <option> list of matching cities.

```
<?php

include "db.php";

$state_id = (int)($_POST['state_id'] ?? 0);

echo '<option value="">Select City</option>';

if ($state_id <= 0) {
    exit;
}

$sql = "SELECT * FROM cities WHERE state_id=$state_id ORDER BY city_name";
$result = mysqli_query($conn, $sql);
```

```

while ($city = mysqli_fetch_assoc($result)) {
    echo '<option value="' . $city['id'] . '">';
    echo htmlspecialchars($city['city_name']);
    echo '</option>';
}

?>

```

9. Client-Side AJAX — js/script.js

js/script.js

This file contains only the dependent State → City dropdown logic. There is no AJAX anywhere else in the project — Add, Edit, Delete and List all use normal page loads and form submissions.

```

$(document).ready(function () {

    $("#state_id").change(function () {
        var state_id = $(this).val();

        $("#city_id").html('<option value="">Loading...</option>');

        if (state_id == "") {
            $("#city_id").html('<option value="">Select City</option>');
            return;
        }

        $.ajax({
            url: "cities.php",
            type: "POST",
            data: { state_id: state_id },
            success: function (response) {
                $("#city_id").html(response);
            },
            error: function () {
                $("#city_id").html('<option value="">Error loading cities</option>');
            }
        });
    });

});

```

10. Uploads Folder

Inside the project root, create an uploads folder that PHP can write to:

```
php-crud/  
|-- index.php  
|-- add.php  
|-- edit.php  
|-- delete.php  
|-- cities.php  
|-- db.php  
|-- uploads/  
`-- js/  
    |-- script.js
```

Note: Make sure PHP has permission to write to the uploads folder (e.g. `chmod 755` or `775` on Linux/macOS).

11. How the State to City AJAX Works

This is the only AJAX portion of the entire application.

11.1 Initial state

```
State  
[ Select State ]  
  
City  
[ Select City ]
```

11.2 User selects a state

```
State  
[ Maharashtra ]
```

11.3 JavaScript catches the change event

```
$("#state_id").change(function () {  
    ...  
});
```

11.4 Request is sent to cities.php

```
$.ajax({  
    url: "cities.php",  
    type: "POST",  
    data: { state_id: state_id }  
});
```

11.5 cities.php receives and queries

```
$state_id = (int)$_POST['state_id'];  
  
SELECT *  
FROM cities  
WHERE state_id = 2  
ORDER BY city_name
```

This returns Mumbai, Pune, and Nagpur, and JavaScript inserts them into the <select id="city_id"> element:

```
State
+-----+
| Maharashtra      v |
+-----+

City
+-----+
| Select City      v |
+-----+
| Mumbai          |
| Pune            |
| Nagpur          |
+-----+
```

12. CRUD Does NOT Use AJAX

This is the key distinction of the project: every CRUD operation is a traditional PHP page load, and only the city dropdown uses AJAX.

12.1 Create

```
index.php -> POST -> add.php -> INSERT -> index.php
```

12.2 Read

```
index.php -> SELECT -> Display table
```

12.3 Update

```
index.php -> edit.php?id=5 -> Show existing data
                                -> POST -> UPDATE -> index.php
```

12.4 Delete

```
index.php -> delete.php?id=5 -> DELETE -> index.php
```

12.5 Dependent dropdown (the one AJAX feature)

```
State selected -> AJAX -> cities.php -> SELECT cities -> City dropdown
```

Note: Only one feature uses AJAX: the State to City dependent dropdown. Everything else is traditional PHP procedural CRUD. Recommendation: in a future version, separate edit.php's POST update logic from its HTML, and add basic server-side image validation (jpg/jpeg/png only, plus a file size limit). This keeps the code beginner-friendly while avoiding common PHP CRUD mistakes.

Appendix: Understanding include in PHP

In this project, include is used to bring another PHP file's code into the current file — most importantly, the database connection.

A.1 Example from the Project

index.php contains:

```
<?php
include "db.php";
?>
```

Here, PHP loads the code from db.php and inserts it logically at that point. db.php itself contains:

```
<?php

$conn = mysqli_connect("localhost", "root", "", "php_crud");

if (!$conn) {
    die("Database connection failed");
}
?>
```

So when index.php runs include "db.php";, PHP creates the \$conn connection, and index.php can immediately use it:

```
$result = mysqli_query($conn, "SELECT * FROM users");
```

A.2 Why Use include?

Without include, the database connection code would need to be repeated in every file: index.php, add.php, edit.php, delete.php, and cities.php. That would be repetitive. Instead:

```
db.php
|
database connection

index.php  --+
add.php    |
edit.php   +-- include db.php
delete.php |
cities.php --+
```

This keeps the project cleaner and avoids duplicated connection code.

A.3 include vs. require

Statement	If the file is missing
include "db.php";	PHP gives a warning; the script may continue running.
require "db.php";	PHP gives a fatal error; the script stops immediately.

For a database connection, many developers prefer require, because the page cannot function at all without it.

A.4 include_once and require_once

PHP also provides `include_once` and `require_once`, which include the file only once even if the statement is reached again during the same request. For this beginner CRUD project, plain `include "db.php";` is perfectly fine.

Note: `include` is not a module import like in Java or Python. It simply tells PHP: "Load and execute the PHP code from this file here." In this project, it is mainly used to reuse the database connection code across every page.